

Strings: analisar e transformar texto

Atividade 1 · Uma sequência imutável

Questões de autoavaliação

1. Registrar as saídas e explicar a construção da nova palavra. Distinguir reatribuir uma variável e alterar uma string.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Original: P n; yth; nohtyP; Original: Python; Novo: Jython. Com Gato e R: G o; ato; otaG; Original: Gato; Novo: Rato. len(texto) é 4.

A expressão 'R' + texto[1:] concatena uma letra com a fatia restante. Criar uma nova string e atribuí-la a uma variável não modifica a string original.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Atividade 2 · Normalizar para comparar

Questões de autoavaliação

1. Comparar in e find. Explicar por que o termo de pesquisa também necessita de normalização.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Original normalizado: 'ana silva'. A presença de ana é True. find('Silva') devolve -1. Mudando para find('silva'), devolve 4. Com 'ANA SOUSA ', o nome é 'ana sousa': ana continua presente, mas silva não é encontrado (-1).

A pesquisa distingue maiúsculas de minúsculas. Tanto o texto como o termo devem seguir a mesma normalização para uma comparação sem distinção de caixa.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Strings: analisar e transformar texto

Atividade 3 · Contar palavras e caracteres

Questões de autoavaliação

1. Explicar o efeito dos espaços nos resultados e por que útil. conserva o ponto dentro do token.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

Lista original: ['python', 'é', 'útil.', 'python', 'é', 'claro.']. Existem seis tokens e duas ocorrências de python. Espaços adicionais não mudam estes resultados, porque split() sem argumento agrupa espaços em branco.

Uma string só com espaços produz uma lista vazia, zero palavras e zero ocorrências. A pontuação continua nos tokens: split separa por espaços, não remove pontos.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Atividade 4 · Interpretar um registo simples

Questões de autoavaliação

1. Registrar os três resultados e distinguir validação da estrutura e validação da nota.
2. Indicar um teste adicional relevante, o resultado previsto e a justificação da escolha.

Resolução e verificação

'102; Rui ;12' produz (102, 'Rui', 12.0). '103;Eva' mostra Número de campos inválido. '104;Leo;25' deve ser rejeitada por nota fora do intervalo.

```
linha = '104;Leo;25'
campos = linha.split(';')
if len(campos) == 3:
    nota = float(campos[2])
    if 0 <= nota <= 20:
        aluno = (int(campos[0]), campos[1].strip(), nota)
        print(aluno)
    else:
        print('Nota inválida')
else:
    print('Número de campos inválido')
```

A estrutura verifica a existência de três campos; o intervalo verifica uma regra sobre o valor convertido. Assume-se conteúdo numérico válido quando há três campos, como indica a atividade.

O teste adicional pode variar. Deve respeitar o domínio da atividade, ter um resultado previsto explícito e verificar um caso distinto dos anteriores. A justificação e a comparação com a execução fazem parte da autoavaliação.

Strings: analisar e transformar texto

Atividade 5 · Aplicação integrada

Questões de autoavaliação

1. Explicar a organização do algoritmo e as decisões tomadas.
2. Comparar resultados previstos e observados em todos os testes pedidos.
3. Identificar uma falha possível e indicar como um teste a poderia revelar.

Resolução e verificação

```
def pesquisar_nomes(nomes, termo):  
    termo = termo.strip().lower()  
    if not termo:  
        return []  
    encontrados = []  
    for nome in nomes:  
        if termo in nome.lower():  
            encontrados.append(nome)  
    return encontrados
```

```
nomes = ['Ana Silva', 'Bruno', 'Mariana', 'Rui']  
termo = ' ANA '  
print(pesquisar_nomes(nomes, termo))  
print(nomes)
```

'ANA ': ['Ana Silva', 'Mariana']; 'rui': ['Rui']; 'xyz' e apenas espaços: []. Mariana contém a sequência ana no fim. A pesquisa usa nomes normalizados apenas na comparação; conserva a grafia original na nova lista. nomes não sofre alterações.

Strings: analisar e transformar texto

Atividade 6 · Contar palavras completas

Questões de autoavaliação

1. Por que o resultado inicial não corresponde à contagem de palavras?
2. Quais são as contagens corretas nos três testes?
3. Que limitação permanece se houver pontuação diferente da prevista?

Resolução e verificação

```
texto = 'Ana, anabela e ANA! Banana.'  
normalizado = texto.lower()  
for sinal in ',.!:':  
    normalizado = normalizado.replace(sinal, ' '  
quantidade = 0  
for palavra in normalizado.split():  
    if palavra == 'ana':  
        quantidade += 1  
print(quantidade)
```

count devolve 4: Ana, anabela, ANA e Banana contêm a substring; as ocorrências sobrepostas não são contadas duas vezes. Palavras completas: 2,2,0. Pontuação não incluída, como ponto e vírgula, pode ficar ligada ao token; a regra de limpeza deve ser ampliada se o domínio exigir.

Strings: analisar e transformar texto

Atividade 7 · Validar um identificador

Questões de autoavaliação

1. Por que o comprimento correto não basta?
2. Que resultado se espera para AB-123, ab-123, A1-123, AB-12, AB_123 e espaços à volta de AB-123?
3. Por que verificar o comprimento antes de aceder à posição 2?

Resolução e verificação

```
def codigo_valido(texto):
    texto = texto.strip()
    if len(texto) != 6:
        return False
    if texto[2] != '-':
        return False
    for c in texto[:2]:
        if c not in 'ABCDEFGHIJKLMNOPQRSTUVWXYZ':
            return False
    for c in texto[3:]:
        if c not in '0123456789':
            return False
    return True

for codigo in ['AB-123', 'ab-123', 'A1-123',
              'AB-12', 'AB_123', ' AB-123 ']:
    print(repr(codigo), codigo_valido(codigo))
```

Resultados: True,False,False,False,False,True. O comprimento não verifica o conteúdo nem o separador. Validá-lo primeiro impede IndexError em strings curtas. Usar conjuntos explícitos de caracteres restringe o formato a ASCII, ao contrário de alguns métodos gerais de classificação Unicode.

Strings: analisar e transformar texto

Atividade 8 · Interpretar e validar registos textuais

Questões de autoavaliação

1. Quais os registos válidos e inválidos no conjunto inicial?
2. Por que só se deve converter depois de validar os caracteres?
3. Que diferença existe entre erro de estrutura, conteúdo e intervalo?

Resolução e verificação

```
def so_algarismos(texto):
    if not texto:
        return False
    for c in texto:
        if c not in '0123456789':
            return False
    return True

def interpretar(linha):
    campos = [c.strip() for c in linha.split(';')]
    if len(campos) != 3:
        return None
    numero, nome, nota = campos
    if not so_algarismos(numero) or not nome:
        return None
    if not so_algarismos(nota):
        return None
    if not 0 <= int(nota) <= 20:
        return None
    return (int(numero), nome, int(nota))

linhas = ['101; Ana ;16', '102;Rui;21',
          '103;;12', '104;Eva', '105;Leo;abc']
validos = []
invalidos = []
for linha in linhas:
    registo = interpretar(linha)
    if registo is None:
        invalidos.append(linha)
    else:
        validos.append(registo)
print(validos)
print(invalidos)
```

Só (101,Ana,16) é válido no conjunto inicial. Os restantes falham por nota fora do intervalo, nome vazio, falta de campo e nota não numérica. Linha vazia devolve None; 106; Inês ;0 devolve (106,Inês,0). A validação de caracteres antecede int; não se aceitam sinais ou decimais neste formato.

Critério de autoavaliação: 2 pontos quando a implementação cumpre as regras, passa os testes pedidos e as respostas explicam os resultados; 1 ponto se falta parte da solução, dos testes ou da justificação; 0 pontos se a tarefa permanece por resolver. Aplicar o mesmo critério às restantes atividades, num total de 16 pontos. A pontuação orienta o estudo e não constitui classificação formal.